

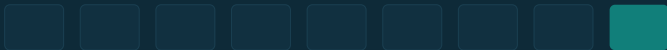


STREAM PROCESSING AVEC APACHE KAFKA

Fenêtrage & jointures

Agréger dans le temps et enrichir des flux

ESGI · 5^e année · Data Engineering · 2025–2026



Objectifs

- **Agréger par tranche de temps**

Fenêtres *tumbling*, *hopping*, *sliding*, *session*.

- **Gérer les données en retard**

Le *grace period* : jusqu'où accepter les enregistrements tardifs.

- **Enrichir un flux (KStream-KTable)**

Une recherche dans une table de référence, à chaque événement.

- **Corréler deux flux (KStream-KStream)**

Une jointure *fenêtrée*, et l'exigence de *co-partitionnement*.

SÉANCE 12

Au programme

01

Fenêtrage

temps, types, grace period

02

Jointures

enrichir, corréler, co-partition

03

Atelier

fenêtrer et enrichir

01

- PARTIE 1

Fenêtrage

Quel temps fait foi ?

- Une agrégation non fenêtrée grandit **indéfiniment**. Souvent on veut un résultat **par tranche de temps** : par minute, par heure. . .
- Kafka Streams raisonne par défaut sur le **temps de l'événement** (l'horodatage du message), pas sur le temps de traitement : les retards et le désordre sont donc la norme.

■ Temps de l'événement

L'instant où le fait s'est produit (porté par le message).

■ Temps de traitement

L'instant où Streams le traite — sujet aux délais réseau.

Quatre façons de découper le temps



■ Tumbling

Taille fixe, sans recouvrement (ci-dessus).

■ Hopping

Taille fixe qui avance par pas → fenêtres qui se chevauchent.

■ Sliding

Fenêtre glissante alignée sur les horodatages.

■ Session

Taille variable, fermée après une période d'inactivité.

Fenêtrer une agrégation

Compter par cle, sur des fenetres d une minute (+ grace)

```
byKey.groupByKey(Grouped.with(Serdes.String(), Serdes.String()))  
  .windowedBy(TimeWindows.ofSizeAndGrace(  
    Duration.ofMinutes(1), Duration.ofSeconds(10)))  
  .count(Materialized.as("windowed-count"))  
  .toStream()  
  .map((wk, n) -> KeyValue.pair(  
    wk.key() + "@" + wk.window().start(), String.valueOf(n)));
```

La clé du résultat est un `Windowed<K>` : une clé *et* une fenêtre (start, end).

Voir les fenêtres compter

Lancer, produire, lire les comptages fenetres

```
mvn -q compile exec:java \  
  -Dexec.mainClass=fr.esgi.kafka.tp12.WindowAndJoin &  
echo 'o-1;19.9;eur' | kafka-console-producer.sh \  
  --bootstrap-server localhost:29092 --topic orders  
kafka-console-consumer.sh --bootstrap-server localhost:29092 \  
  --topic orders-windowed-count --property print.key=true
```

■ Clé fenêtrée

La sortie est indexée par
devise@début-de-fenêtre.

■ Par tranche

Le compteur d'une devise repart à chaque
nouvelle fenêtre.

02

- PARTIE 2

Jointures

Relier deux sources

Jointure	Usage	Fenêtrée ?
KStream-KTable	enrichir un événement (lookup)	non
KStream-KStream	corrélér deux flux d'événements	oui
KTable-KTable	combiner des tables de référence	non
KStream-GlobalKTable	lookup global (sans co-partition)	non

Co-partitionnement : mêmes clés *et* même nombre de partitions des deux côtés (sauf GlobalKTable).

Enrichir, puis corréler

```
KStream-KTable (lookup) puis KStream-KStream (fenetre)
```

```
// Enrichir chaque commande par le taux de sa devise
ordersByCcy.join(rates,
    (order, rate) -> convert(order, rate))
    .to("orders-converted");

// Corréler deux flux sur une fenetre de 30 minutes
clicks.join(buys, (c, b) -> attribution(c, b),
    JoinWindows.ofTimeDifferenceAndGrace(
        ofMinutes(30), ofMinutes(5)),
    StreamJoined.with(Serdes.String(), sClick, sBuy));
```

Variantes : `join(inner)`, `leftJoin`, `outerJoin`. Le `ValueJoiner` calcule le résultat.

Enrichir par une table de référence

Charger la table de taux, lire les commandes converties

```
# Table de reference : devise -> taux (cle = devise)
printf 'EUR:1.0\nUSD:0.92\nGBP:1.15\n' | \
  kafka-console-producer.sh --bootstrap-server localhost:29092 \
    --topic rates --property parse.key=true --property key.separator=:
# Produire 'o-9;50;usd' dans orders, puis lire la conversion
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-converted --from-beginning
```

■ Lookup

Chaque commande lit le taux **courant** de sa devise.

■ Vivante

Mettre à jour un taux change les conversions suivantes.

03

- PARTIE 3

Atelier — fenêtrer et enrichir

Cluster et topics

Demarrer le cluster et creer les topics

```
docker compose up -d                # 3 brokers KRaft

for t in orders rates orders-converted orders-windowed-count; do
    kafka-topics.sh --bootstrap-server localhost:29092 \
        --create --topic $t --partitions 3 --replication-factor 3
done
```

■ Entrées

orders (flux) et rates (table de référence, clé = devise).

■ Co-partition

Tous en 3 partitions : la jointure est possible.

Construire fenêtrage et jointure

- 1 Lire `orders` (flux) et `rates` en `KTable` ; `selectKey` sur la devise.
- 2 **Fenêtrer** : `windowedBy + count` → `orders-windowed-count`.
- 3 **Joindre** : `KStream-KTable` avec `rates` → `orders-converted`.
- 4 Charger `rates`, produire des commandes, lire les deux sorties.
- 5 Mettre à jour un taux et observer l'effet sur les conversions suivantes.

Fenêtres et enrichissement

Lire les deux sorties

```
# Comptages par devise et par fenetre
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-windowed-count --property print.key=true
# Commandes converties (montant en devise de base)
kafka-console-consumer.sh --bootstrap-server localhost:29092 \
  --topic orders-converted --from-beginning
```

■ Fenêtré

Les comptages sont indexés par
devise@fenêtre.

■ Enrichi

Chaque commande sort avec son montant
converti.

Pour aller plus loin

Objectif — éprouver les fenêtres et les variantes de jointure.

■ 1. **suppress**

N'émettre que le résultat **final** de chaque fenêtre (`suppress`).

■ 2. **leftJoin**

Garder les commandes sans taux connu via `leftJoin`.

■ 3. **Session**

Compter par **session** d'activité au lieu d'une fenêtre fixe.

Exploiter les fenêtres et l'état

- **Résultats finaux** : `suppress` n'émet qu'à la fermeture d'une fenêtre, pour des sorties propres.
- **Interroger l'état** : les **Interactive Queries** (séance 14) liront ces stores fenêtrés sans repasser par un topic.
- **Alternatives** : `ksqlDB` exprime fenêtres et jointures en SQL ; Apache `Flink` pour d'autres compromis.

■ Acquis

Agréger dans le temps et enrichir des flux.

À venir

Affiner le retard (S13),
puis exposer l'état (S14).

SÉANCE 12

À retenir

- ✓ Fenêtrer borne une agrégation dans le **temps de l'événement** ; clé du résultat = `Windowed<K>`.
- ✓ Le **grace period** fixe jusqu'où accepter les données en retard.
- ✓ Quatre fenêtres : *tumbling*, *hopping*, *sliding*, *session*.
- ✓ `KStream-KTable` enrichit (lookup) ; `KStream-KStream` corrèle (fenêtré).
- ✓ Toute jointure exige le **co-partitionnement** (sauf `GlobalKTable`).



LA SUITE — SÉANCE 13

Correction & données en retard

Revenir sur l'atelier fenêtrage/jointures et maîtriser le grace period, l'ordre et les horodatages.